

LAPORAN PRAKTIKUM
IF1221 LOGIKA KOMPUTASIONAL



Permainan Kartu UNI

Disusun oleh :

Justin William (13525042)

Muhammad Ridwan Nasir Firdaus (13525096)

Jeremy Gerald Sutanto (13525104)

Christian Immanuel (13525116)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
BANDUNG
2026

DAFTAR ISI

LAPORAN PRAKTIKUM.....	0
DAFTAR ISI.....	1
1. Perintah dan Hasil Eksekusi Program.....	2
1.1. Perintah: startGame.....	2
1.2. Perintah: mainkanKartu(NomorKartu), cekInfo, lihatKartu.....	5
1.3. Perintah: ambilKartu.....	13
1.4. Perintah: tantang.....	13
1.5. Perintah: uni(X).....	15
1.6. Perintah: tangkap>NamaPemain).....	16
1.7. Perintah: lihatCommand.....	17
1.8. Perintah: saveGame dan loadGame.....	18
1.9. Perintah: godsHand.....	20
1.10. Perintah: sembunyikanKartu.....	21
1.11. Perintah: endGame.....	22
1.12. Sistem: Hitung Poin Pemenang.....	23
Pembagian Kerja.....	25
SURAT PERNYATAAN PENGGUNAAN KECERDASAN BUATAN/AI.....	26

1. Perintah dan Hasil Eksekusi Program

1.1. Perintah: startGame

a) Penjelasan kegunaan perintah.

Perintah startGame digunakan untuk memulai permainan UNI dari kondisi awal. Perintah ini melakukan seluruh proses inisialisasi permainan, mulai dari, meminta jumlah pemain, meminta nama pemain, mengacak urutan pemain, membuat dan mengacak deck kartu, membagikan 7 kartu kepada setiap pemain, menentukan kartu awal di discard pile, lalu menetapkan pemain pertama yang mendapat giliran. Perintah ini juga akan menghapus state game sebelumnya.

b) Skenario beserta contoh penggunaannya.

Berikut adalah contoh skenario pengujian perintah:

```
GNU Prolog 1.4.5 (64 bits)
Compiled Feb 23 2020, 20:14:50 with gcc
By Daniel Diaz
Copyright (C) 1999-2020 Daniel Diaz
| ?- startGame.
Masukkan jumlah pemain: 5.

Mohon masukkan angka antara 2 - 4.
Masukkan jumlah pemain: 1.

Mohon masukkan angka antara 2 - 4.
Masukkan jumlah pemain: 3.
Masukkan nama pemain 1: a.
Masukkan nama pemain 2: b.
Masukkan nama pemain 3: a.
Nama sudah digunakan. Masukkan nama lain: ''.
Nama tidak boleh kosong. Masukkan nama lain: c.

Urutan pemain: b - c - a
Setiap pemain mendapatkan 7 kartu acak.
Kartu discard top: merah-8
Giliran b.

true ?

(2 ms) yes
| ?-
```

Perintah dijalankan dengan: ?- startGame. Setelah berhasil, permainan sudah masuk ke state aktif dan siap dilanjutkan ke perintah-perintah permainan berikutnya.

a. Contoh penggunaan valid untuk startGame:

Masukkan jumlah pemain: 3.

Masukkan nama pemain 1: a.

Masukkan nama pemain 2: b.

Masukkan nama pemain 3: c.

Pada contoh tersebut, input valid karena:

- Jumlah pemain adalah 3, berada dalam rentang valid, yaitu 2 sampai 4.
- Nama pemain tidak kosong.
- Nama pemain tidak duplikat.
- Game berhasil membuat urutan pemain, membagikan kartu, dan menentukan giliran pertama.

b. Contoh penggunaan tidak valid untuk startGame:

1. Jumlah pemain tidak valid.

Masukkan jumlah pemain: 5.

Masukkan jumlah pemain: 1.

Input 5 dan 1 tidak valid karena UNO pada program ini hanya menerima 2 sampai 4 pemain. Program tidak langsung gagal, tetapi meminta input ulang sampai pengguna memasukkan angka yang valid.

2. Duplikasi Nama pemain

Masukkan nama pemain 1: a.

Masukkan nama pemain 2: b.

Masukkan nama pemain 3: a.

Nama a pada pemain ketiga tidak valid karena sudah digunakan oleh pemain pertama. Dalam tes ini, program meminta user untuk memasukan nama lain hanya untuk pemain 3. Program baru menerima input setelah pengguna memasukkan nama valid, yaitu c.

3. Nama pemain kosong

Masukkan nama pemain 1: ‘’.

Nama pemain tidak valid karena input tidak boleh kosong, sehingga program akan terus meminta input ulang sampai pengguna memasukkan nama dengan format yang tepat dan tidak duplikat.

c) Dampak terhadap jalannya permainan.

Perintah `startGame` mengubah state permainan secara besar karena perintah ini membentuk seluruh state awal permainan. Sebelum game dimulai, program terlebih dahulu memanggil `reset_game_state`, yaitu proses untuk menghapus seluruh state permainan sebelumnya agar data game lama tidak tercampur dengan game baru.

Pada proses `startGame`, sistem akan:

1. Mengacak deck kartu.
2. Membagikan 7 kartu kepada setiap pemain.
3. Mengambil satu kartu awal untuk diletakkan pada discard pile.
4. Menentukan pemain pertama dan arah permainan awal.
5. Menyimpan seluruh state awal permainan ke dalam dynamic predicate.
6. Menampilkan ringkasan awal permainan, seperti urutan pemain, kartu discard teratas, dan pemain yang mendapatkan giliran pertama.

Akibat dari proses tersebut, state permainan berubah dari kondisi belum dimulai menjadi kondisi permainan aktif. Seluruh komponen penting permainan, seperti deck, tangan pemain, discard pile, giliran pemain, dan arah permainan sudah terbentuk sehingga permainan dapat berjalan.

1.2. Perintah: mainkanKartu(NomorKartu), cekInfo, lihatKartu.

a) Penjelasan kegunaan perintah.

Perintah mainkanKartu digunakan untuk memainkan tepat satu kartu berdasarkan nomor urut pada daftar kartu yang dimilikinya. Namun, kartu yang dimainkan harus valid, yaitu memiliki warna yang sama, angka yang sama atau merupakan kartu wild yang bisa dimainkan kapan saja. Indeks kartu pemain dimulai dari 1.

Perintah cekInfo digunakan untuk melihat informasi saat ini termasuk kartu teratas pada discard pile, urutan giliran pemain, nama pemain dan jumlah kartu yang dimiliki setiap pemain.

Perintah lihatKartu digunakan untuk melihat daftar kartu yang sedang dimiliki oleh pemain yang sedang mendapat giliran termasuk kartu yang disembunyikan beserta nomor urutnya. Perintah lihatKartu dan cekInfo akan menyesuaikan informasi tim dalam mod turnamen.

b) Skenario beserta contoh penggunaannya.

Berikut adalah contoh skenario pengujian perintah:

```

| ?- cekInfo.

Kartu discard top: biru-8
Warna aktif: biru
Arah permainan: kanan
Urutan pemain: c - a - b

Nama pemain 1: b
Jumlah kartu : 6

Nama pemain 2: c
Jumlah kartu : 7

Nama pemain 3: a
Jumlah kartu : 7

true ?

yes
| ?- lihatKartu.

Berikut kartu yang anda miliki.
1. kuning-3
2. biru-4
3. biru-1
4. hitam-wild_draw_four (disembunyikan)
5. hitam-wild (disembunyikan)
6. hitam-mimic (disembunyikan)
7. merah-1

true ?

yes
| ?- mainkanKartu(1).

Kartu tersebut tidak valid untuk dimainkan saat ini.

Nomor urut kartu harus berupa bilangan bulat positif.

no
| ?- mainkanKartu(2).

c memainkan kartu: biru-4.

Giliran a.

true ?

```

Perintah cekInfo dijalankan dengan: ?- cekInfo. Setelah berhasil program akan menampilkan informasi permainan yang sedang berlangsung. Informasi yang ditampilkan adalah kartu discard top, warna aktif, arah permainan, urutan pemain, dan nama serta jumlah kartu tiap pemain.

Perintah lihatKartu dijalankan dengan ?- lihatKartu. Setelah berhasil program akan menampilkan kartu yang dimiliki pemain yang sedang mendapatkan giliran beserta dengan nomor urut tiap kartu.

Perintah `mainkanKartu` dijalankan dengan `?- mainkanKartu(X)`. Dimana `X` adalah nomor urut kartu yang ingin dimainkan oleh pemain yang sedang mendapat giliran. Setelah valid dan berhasil, kartu yang dipilih akan dimainkan dan menjadi kartu discard top baru, dan kartu yang dimiliki pemain tersebut akan berkurang 1.

a) Contoh penggunaan valid untuk `mainkanKartu`:

Berdasarkan perintah `cekInfo` dan `lihatKartu` dari contoh screenshot, kartu discard top saat ini adalah biru-8.

`?- mainkanKartu(2)`

memainkan kartu: biru-4

Contoh tersebut valid karena warnanya sama dengan warna aktif saat ini. Kartu yang dimainkan harus memenuhi salah satu kondisi berikut:

- Warna kartu sama dengan warna aktif
- Angka/symbol sama dengan kartu discard top
- Kartu wild

b) Contoh penggunaan tidak valid untuk `mainkanKartu`:

`?- mainkanKartu(1)`

Contoh tersebut tidak valid karena warnanya berbeda dengan warna aktif saat ini. Selain itu, angka/symbol kartu tersebut juga tidak sama dengan angka/symbol kartu discard top, sehingga kartu tersebut tidak valid untuk dimainkan. Kalau tidak valid, program akan meminta pemain mengeluarkan kartu yang valid atau meminta pemain mengambil kartu jika tidak punya.

`?- mainkanKartu(-1)`

Contoh tersebut tidak valid karena nomor urutnya negatif, sedangkan nomor urut kartu harus positif, sehingga program akan menampilkan pesan tidak valid berupa nomor urut harus bilangan positif dan akan meminta pemain memasukkan input kembali.

Dalam perintah mainkanKartu ini terdapat beberapa skenario unik yang terjadi ketika pemain mengeluarkan kartu aksi, kartu aksi tersebut akan memicu skenario sebagai berikut:

- Kartu Skip

```
| ?- lihatKartu.  
  
Berikut kartu yang anda miliki.  
1. merah-7  
2. merah-skip  
3. kuning-5  
4. merah-9  
  
true ?  
  
yes  
| ?- mainkanKartu(2).  
  
b memainkan kartu: merah-skip.  
  
c kehilangan giliran.  
  
Giliran a.  
  
true ?
```

Kartu skip akan menghasilkan efek berupa pemain berikutnya akan dilewati dan giliran akan berpindah ke pemain setelahnya.

- Kartu Reverse

```

| ?- mainkanKartu(6).
Pilih warna aktif baru (merah/kuning/hijau/biru): kuning.
Warna aktif berubah menjadi kuning.

c memainkan kartu: hitam-wild.

Giliran a.

true ?

yes
| ?- lihatKartu.

Berikut kartu yang anda miliki.
1. kuning-1
2. kuning-reverse
3. hijau-7
4. hijau-2

true ?

yes
| ?- mainkanKartu(2).

a memainkan kartu: kuning-reverse.
Arah permainan berubah menjadi kiri.

Giliran c.

true ? █

```

Efek dari kartu reverse ini adalah membalikkan giliran dalam permainan, semisal yang awalnya searah jarum jam maka akan menjadi berlawanan arah dan begitu juga sebaliknya.

- Kartu drawTwo

```

| ?- mainkanKartu(5).

a memainkan kartu: hijau-draw_two.

b harus mengambil 2 kartu dan kehilangan giliran.

Giliran b.

true ?
Action (; for next solution, a for all solutions, RET to stop) ?

yes
| ?- ambilKartu.

b mendapatkan kartu: hitam-wild + biru-0.

Giliran a.

```

Efek dari kartu *draw two* ini adalah memicu penambahan kartu sekaligus melompati giliran pemain berikutnya. Pemain yang terkena efek kartu ini diwajibkan untuk mengambil secara acak 2 kartu dari *deck* ke tangan mereka, serta kehilangan hak untuk bermain pada putaran tersebut.

- Kartu Wild

```
| ?- lihatKartu.  
Berikut kartu yang anda miliki.  
1. merah-7  
2. merah-skip  
3. kuning-5  
4. merah-9  
5. hitam-wild (disembunyikan)  
  
true ?  
  
yes  
| ?- mainkanKartu(5).  
Pilih warna aktif baru (merah/kuning/hijau/biru): merah.  
Warna aktif berubah menjadi merah.  
  
b memainkan kartu: hitam-wild.  
  
Giliran c.
```

Efek dari kartu *wild* ini adalah memberikan kebebasan kepada pemain aktif untuk mengatur jalannya permainan. Kartu ini dapat dikeluarkan kapan saja tanpa terpengaruh oleh warna aktif di lapangan, dan setelahnya pemain wajib memilih warna baru (merah, kuning, hijau, atau biru) sebagai warna untuk giliran selanjutnya.

- Kartu Wild Draw Four

```

Berikut kartu yang anda miliki.
1. merah-7
2. merah-skip
3. kuning-5
4. hitam-wild_draw_four (disembunyikan)
5. merah-9
6. hitam-wild (disembunyikan)

true ?

yes
| ?- mainkanKartu(4).
Pilih warna aktif baru (merah/kuning/hijau/biru): biru.
Warna aktif berubah menjadi biru.

b memainkan kartu: hitam-wild_draw_four.

c dapat memilih ambilKartu atauantang.

Giliran c.

true ?

yes
| ?- lihatCommand.

Aksi utama yang tersedia:
1. ambilKartu
2. antang

Aksi pendukung yang tersedia:
1. lihatCommand
2. lihatKartu
3. cekInfo
4. tampilkanKartu
5. saveGame
6. loadGame

yes
| ?- ambilKartu.

c mendapatkan kartu: kuning-2 + kuning-8 + hitam-wild + hijau-3.

Giliran a.

```

Pemain yang mengeluarkan kartu ini berhak menentukan warna aktif baru untuk permainan, sekaligus memaksa pemain berikutnya di dalam urutan giliran untuk mengambil 4 kartu dari *deck* dan kehilangan hak gilirannya.

- Kartu Mimic

```
Berikut kartu yang anda miliki.  
1. kuning-3  
2. biru-1  
3. hitam-wild_draw_four (disembunyikan)  
4. hitam-wild (disembunyikan)  
5. hitam-mimic (disembunyikan)  
6. kuning-2  
7. kuning-8  
8. hitam-wild (disembunyikan)  
9. hijau-3  
  
true ?  
  
(1 ms) yes  
| ?- mainkanKartu(5).  
  
c memainkan kartu: hitam-mimic.  
  
Menelusuri riwayat permainan.  
Kartu aksi terakhir yang dimainkan: kuning-skip (oleh a, 2 giliran lalu)  
Kartu mimic menyalin efek skip!  
Pilih warna aktif baru (merah/kuning/hijau/biru): merah.  
Warna aktif sekarang: merah.  
  
a kehilangan giliran.  
  
Giliran b.
```

Efek dari kartu mimic ini ialah dapat menirukan kartu aksi yang telah dimainkan, dan jika belum ada kartu aksi, maka kartu ini akan memiliki sifat seperti kartu wild.

c) Dampak terhadap jalannya permainan.

Perintah mainkanKartu(X) akan mengubah state permainan hanya jika kartu ke-X valid untuk dimainkan. Kartu dianggap valid jika sesuai dengan warna aktif, angka/symbol discard top, atau merupakan kartu wild. Perintah cekInfo dan lihatKartu tidak mengubah state permainan secara signifikan karena hanya menampilkan info.

1.3. Perintah: ambilKartu

a) Penjelasan kegunaan perintah.

Perintah ini digunakan untuk mengambil kartu dari deck ketika pemain tidak dapat atau tidak ingin memainkan kartu.

Berikut adalah contoh skenario pengujian perintah:

```
Giliran a.  
  
true ?  
  
yes  
| ?- ambilKartu.  
  
a mendapatkan kartu: merah-5.  
  
Giliran b.  
  
yes  
| ?-
```

b) Dampak terhadap state permainan

Jumlah kartu pemain bertambah, lalu giliran berpindah ke pemain berikutnya. Selain itu, perintah ini juga menampilkan informasi kartu yang diperoleh pemain, yaitu berupa nomor kartu dan simbol/jenis kartu yang didapat dari deck

1.4. Perintah: tantang

a) Penjelasan kegunaan perintah.

Perintah tantang digunakan untuk menantang pemain sebelumnya setelah ia memainkan kartu `wild_draw_four`. Tantangan dilakukan apabila pemain berikutnya merasa bahwa pemain sebelumnya sebenarnya masih memiliki kartu lain yang dapat dimainkan, sehingga penggunaan `wild_draw_four` dianggap tidak sah.

Saat perintah ini dijalankan, sistem akan memeriksa kartu di tangan pemain sebelumnya untuk menentukan apakah kartu `wild_draw_four` memang

dimainkan sesuai aturan atau tidak. Hasil pemeriksaan tersebut akan menentukan pemain mana yang menerima penalti kartu tambahan.

Berikut adalah contoh skenario pengujian perintah:

```
?  
| ?- mainkanKartu(8).  
Pilih warna aktif baru (merah/kuning/hijau/biru): merah.  
Warna aktif berubah menjadi merah.  
  
b memainkan kartu: hitam-wild_draw_four.  
  
a dapat memilih ambilKartu atau tantang.  
  
Giliran a.  
  
true ?  
  
yes  
| ?- tantang.  
  
Tantangan dilakukan!  
Memeriksa kartu b...  
Tantangan berhasil. b mendapatkan 4 kartu acak: kuning-reverse + biru-4 + hijau-skip + biru-2.  
  
Giliran b.  
  
true ?
```

```
Giliran a.  
  
yes  
| ?- lihatKartu.  
  
Berikut kartu yang anda miliki.  
1. hijau-5  
2. merah-8  
3. hitam-wild (disembunyikan)  
4. hitam-wild_draw_four (disembunyikan)  
  
true ?  
  
yes  
| ?- mainkanKartu(4).  
Pilih warna aktif baru (merah/kuning/hijau/biru): hijau.  
Warna aktif berubah menjadi hijau.  
  
a memainkan kartu: hitam-wild_draw_four.  
  
b dapat memilih ambilKartu atau tantang.  
  
Giliran b.  
  
true ?  
  
yes  
| ?- tantang.  
  
Tantangan dilakukan!  
Memeriksa kartu a...  
Tantangan gagal. b mendapatkan 6 kartu acak: hijau-8 + hitam-wild_draw_four + merah-0 + biru-1 + biru-7 + biru-9.  
  
Giliran a.  
  
true ?
```

b) Dampak terhadap state permainan

Perintahantang dapat mengubah state permainan tergantung hasil dari proses tantangan. Sistem akan memeriksa kartu yang dimiliki pemain sebelumnya untuk menentukan apakah kartu wild_draw_four dimainkan secara sah atau tidak.

Jika tantangan berhasil, maka pemain yang memainkan wild_draw_four akan menerima penalti berupa pengambilan kartu tambahan, dan state permainan berubah pada jumlah kartu pemain tersebut. Sebaliknya, jika tantangan gagal, maka pemain yang melakukan tantangan akan menerima penalti kartu tambahan sehingga jumlah kartunya bertambah.

1.5. Perintah: uni(X)

a) Penjelasan kegunaan perintah.

Perintah uni digunakan untuk memainkan kartu kedua paling terakhir sekaligus menyerukan UNI setelah kartu pemain bersisa satu. Jika pemain memainkan kartu tinggal bersisa satu tanpa menyerukan atau menggunakan perintah UNI, pemain lain dapat menggunakan perintah tangkap untuk memberikan penalti. Jika perintah tidak valid alias kartu yang dimainkan tidak membuat kartu menjadi bersisa satu maka pemain mendapatkan penalti berupa mendapat 1 kartu acak dan giliran berpindah ke pemain selanjutnya.

Berikut adalah contoh skenario pengujian perintah:

```
| ?- lihatKartu.  
  
Berikut kartu yang anda miliki.  
1. kuning-5  
2. merah-9  
  
true ?  
  
yes  
| ?- uni(2).  
  
b memainkan kartu: merah-9.  
b menyerukan UNI!  
  
Giliran c.  
  
true ?
```


b) Dampak terhadap state permainan.

Perintah uni(X) dapat mengubah state permainan jika kartu ke-X valid untuk dimainkan dan kartu pemain tersebut tepat bersisa satu. Jika perintah valid, maka giliran akan berpindah ke pemain selanjutnya, tetapi kalau tidak valid, maka pemain akan mendapatkan penalti berupa 1 kartu acak. Pemain dengan kartu yang bersisa satu tanpa menggunakan perintah uni bisa ditangkap oleh pemain lain sehingga akan mendapatkan penalti.

1.6. Perintah: tangkap>NamaPemain)

a) Penjelasan kegunaan perintah.

Perintah tangkap digunakan untuk menangkap pemain yang belum menyerukan UNI ketika jumlah kartunya tinggal satu. Jika terdapat pemain yang memiliki satu kartu tetapi belum menggunakan perintah uni, maka pemain lain dapat menjalankan perintah tangkap untuk memberikan penalti sebanyak 2 kartu acak kepada pemain tersebut. Namun, kalau pemain yang dituduh tidak melanggar, maka pemain yang menggunakan perintah tangkap akan mendapatkan penalti sebanyak 1 kartu acak. Perintah ini menjadi hukuman dalam permainan agar pemain harus melakukan perintah UNI dan tangkap pada saat yang tepat.

Berikut adalah contoh skenario pengujian perintah:

```
| ?- lihatKartu.  
  
Berikut kartu yang anda miliki.  
1. biru-5  
2. kuning-2  
  
true ?  
  
yes  
| ?- mainkanKartu(1).  
  
a memainkan kartu: biru-5.  
  
Giliran b.  
  
true ?  
  
yes  
| ?- tangkap(a).  
  
a tertangkap tidak menyerukan UNI.  
a mendapatkan 2 kartu penalti.  
  
Giliran a.  
  
yes  
| ?-
```

b) Dampak terhadap jalan permainan.

Perintah tangkap mengubah state permainan dengan memberikan penalti kepada pemain yang gagal menyerukan UNI saat kartunya tinggal satu. Setelah pemain berhasil ditangkap, jumlah kartu pemain tersebut akan bertambah karena harus mengambil kartu penalti dari deck. Selain perubahan jumlah kartu, status pemain yang sebelumnya hampir menang juga berubah karena ia tidak lagi berada pada kondisi satu kartu.

1.7. Perintah: lihatCommand

a) Penjelasan kegunaan perintah

Perintah lihatCommand digunakan untuk melihat perintah atau aksi yang tersedia untuk pemain jalankan. Terdapat dua bagian yang ditampilkan, yaitu aksi utama dan aksi pendukung. Bagian aksi utama berisi kumpulan perintah yang dapat memengaruhi jalannya permainan, sedangkan aksi pendukung merupakan kumpulan perintah yang hanya menampilkan informasi permainan atau hanya sebagai perintah utilitas.

```
(2 ms) yes
| ?- lihatCommand.

Aksi utama yang tersedia:
1. mainkanKartu(NomorUrutKartuDiTangan)
2. ambilKartu
3. uni(NomorUrutKartuDiTangan)
4. tangkap>NamaPemain)
5. godsHand
6. sembunyikanKartu(NomorUrutKartuDiTangan)

Aksi pendukung yang tersedia:
1. lihatCommand
2. lihatKartu
3. cekInfo
4. tampilkanKartu
5. saveGame
6. loadGame
```

b) Dampak terhadap jalan permainan.

Perintah `lihatCommand` secara tidak langsung berdampak kepada jalannya pertandingan, yaitu melalui pemberian informasi mengenai aksi/perintah yang tersedia untuk pemain jalankan. Melalui informasi yang ditampilkan, pemain dapat dengan cepat mengetahui pilihan perintah yang tersedia tanpa harus mengingat seluruh command yang ada dalam permainan, sehingga membuat proses bermain menjadi lebih lancar dan mengurangi kebingungan, terutama bagi pemain yang baru pertama kali memainkan permainan tersebut.

1.8. Perintah: `saveGame` dan `loadGame`

a) Penjelasan kegunaan perintah.

Perintah `saveGame` digunakan untuk menyimpan kondisi permainan saat ini ke dalam sebuah file dengan format `txt`. Program akan meminta pemain untuk memasukkan nama file yang akan digunakan untuk menyimpan permainan. Kemudian, file akan disimpan dengan path relatif terhadap direktori program. Jika sudah ada file dengan nama yang sama, maka isi file akan ditimpa dengan kondisi permainan terbaru. Perintah ini tidak dapat digunakan ketika pemain diharuskan memilih command tertentu, seperti pemain yang terkena `wild_draw_four` yang memaksa pemain tersebut hanya bisa memainkan `ambilKartu` atau `tantang saja`.

Perintah `loadGame` digunakan untuk memuat kembali kondisi permainan yang sebelumnya sudah disimpan ke dalam file `txt` oleh perintah `saveGame`. Program akan meminta pemain memasukkan nama file yang ingin dimuat. Kalau tidak ditemukan, maka program akan menampilkan pesan kesalahan atau tidak valid.

Berikut adalah contoh skenario pengujian perintah:

```

yes
| ?- saveGame.
Masukkan nama file penyimpanan:

'test1.txt'.

Status permainan berhasil disimpan ke test1.txt.

true ?

yes
| ?-
Prolog interruption (h for help) ? e
jer@Jer:~/prolog/IF1221_G23_dicarryJerZ/src$ ./main
GNU Prolog 1.4.5 (64 bits)
Compiled Feb 23 2020, 20:14:50 with gcc
By Daniel Diaz
Copyright (C) 1999-2020 Daniel Diaz
| ?- loadGame.
Masukkan nama file yang akan dimuat: 'test1.txt'.

Status permainan berhasil dimuat dari test1.txt.
Melanjutkan giliran c.

yes
| ?- cekInfo.

Kartu discard top: kuning-1
Warna aktif: kuning
Arah permainan: kiri
Urutan pemain: c - b - a

Nama pemain 1: b
Jumlah kartu : 1

Nama pemain 2: c
Jumlah kartu : 5

Nama pemain 3: a
Jumlah kartu : 2

true ?

```

b) Dampak terhadap jalan permainan

Perintah `saveGame` menyimpan seluruh kondisi permainan saat ini ke dalam file agar permainan dapat dilanjutkan kembali di waktu lain. Progres permainan tidak hilang meskipun program ditutup atau permainan dihentikan sementara.

Perintah `loadGame` mengubah state permainan secara langsung karena sistem memuat kembali seluruh kondisi permainan dari file penyimpanan. State permainan yang sedang dijalankan diganti dengan state permainan yang ada pada file yang sebelumnya sudah di save.

1.9. Perintah: godsHand

a) Penjelasan kegunaan perintah.

Perintah `godsHand` digunakan untuk menjalankan kejadian acak khusus di dalam permainan. Ketika perintah ini aktif, sistem akan memilih satu pemain secara acak, lalu memilih satu kartu secara acak dari tangan pemain tersebut untuk dipindahkan ke tangan pemain lain yang juga dipilih secara acak.

Berikut adalah contoh skenario pengujian perintah:

```
yes
| ?- lihatKartu.

Berikut kartu yang anda miliki.
1. hitam-mimic (disembunyikan)
2. biru-6
3. kuning-6
4. merah-4
5. kuning-0
6. biru-0

true ?

yes
| ?- godsHand.

Tuhan telah berkehendak.
Kartu kuning-2 milik a berpindah ke tangan b!

Giliran b.

yes
| ?- lihatKartu.

Berikut kartu yang anda miliki.
1. hitam-mimic (disembunyikan)
2. biru-6
3. kuning-6
4. merah-4
5. kuning-0
6. biru-0
7. kuning-2

true ?
```

b) Dampak terhadap jalan permainan

Perintah `godsHand` mengubah state permainan karena terjadi perpindahan kartu antar pemain secara acak. Hal ini mengakibatkan isi tangan pemain berubah, dan jumlah kartu pemain-pemain tertentu dapat berubah.

1.10. Perintah: sembunyikanKartu

a) Penjelasan kegunaan perintah

Perintah sembunyikanKartu digunakan untuk menyembunyikan kartu tertentu yang dimiliki pemain dari tampilan normal permainan. Setelah kartu disembunyikan, kartu tersebut tidak akan ditampilkan secara detail ketika pemain melihat daftar kartunya, melainkan hanya ditandai sebagai kartu tersembunyi.

Berikut adalah contoh skenario pengujian perintah:

```
| ?- lihatKartu.  
  
Berikut kartu yang anda miliki.  
1. hijau-5  
2. merah-8  
3. hitam-wild (disembunyikan)  
  
true ?  
  
yes  
| ?- sembunyikanKartu(1).  
  
Kartu hijau-5 berhasil disembunyikan.  
Giliran b.  
  
true ?
```

b) Dampak terhadap jalan permainan

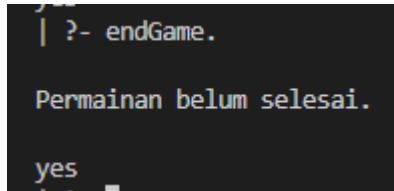
Perintah sembunyikanKartu memberikan dampak pada status visibilitas kartu. Kartu yang dipilih tetap berada di tangan pemain yang sama, tetapi statusnya berubah menjadi tersembunyi. Akibatnya, kartu yang disembunyikan tidak dapat lagi terlihat oleh lawan lain.

1.11. Perintah: endGame

a) Penjelasan kegunaan perintah

Perintah `endGame` digunakan untuk mengakhiri permainan UNI. Perintah ini berfungsi sebagai tanda bahwa permainan sudah selesai dan status permainan tidak digunakan lagi untuk melanjutkan ronde berikutnya.

Berikut adalah contoh skenario pengujian perintah:



```
?- endGame.  
Permainan belum selesai.  
yes
```

b) Skenario beserta contoh penggunaannya

Perintah ini melakukan pengecekan apakah permainan sudah memenuhi syarat untuk diakhiri. Jika permainan masih berjalan dan belum ada pemain yang memenangkan permainan, maka sistem akan menampilkan pesan “Permainan belum selesai.”

a) Contoh penggunaan valid untuk `endGame`:

Salah satu pemain telah berhasil menghabiskan seluruh kartunya. Lalu menjalankan perintah `?- endGame`.

b) Contoh penggunaan tidak valid untuk `endGame`:

Permainan masih berlangsung dan belum ada pemain yang menghabiskan seluruh kartunya. Lalu menjalankan perintah `?- endGame`.

c) Dampak terhadap jalan permainan

Dampak dari `endgame` adalah menghentikan seluruh proses permainan jika kondisi akhir sudah terpenuhi. Setelah perintah ini berhasil dijalankan, permainan langsung diselesaikan, dan state permainan berpindah ke kondisi selesai.

1.12. Sistem: Hitung Poin Pemenang

a) Penjelasan kegunaan

Ketika salah satu pemain menghabiskan kartunya, maka program akan otomatis memanggil perintah endGame untuk menghentikan permainan dan masuk kepada perhitungan poin. Poin dihitung berdasarkan sisa kartu yang dimiliki setiap pemain. Nilai dari tiap kartu adalah sebagai berikut:

- a) Untuk kartu 0-9, nilainya sesuai dengan angka pada kartu, kecuali kartu 0 yang bernilai 1.
- b) Untuk kartu reverse, skip, dan drawtwo nilainya 10.
- c) Untuk kartu wild, wild draw four, dan mimic bernilai 20 poin.

Pemain yang pertama kali menghabiskan kartunya akan mendapat 0 poin, tetapi akan langsung mendapat peringkat pertama alias menjadi pemenang. Pemain sisanya diurutkan berdasarkan jumlah poin yang dimiliki, pemain dengan jumlah poin yang lebih kecil akan mendapatkan peringkat yang lebih tinggi.

Kondisi-kondisi tertentu:

- a) Jika ada dua atau lebih pemain yang memiliki jumlah poin yang sama, maka pemain yang memiliki jumlah kartu lebih sedikit akan menempati peringkat yang lebih tinggi.
- b) Jika jumlah kartu dan jumlah poin juga sama, maka pemain yang urutannya lebih awal akan menempati peringkat yang lebih tinggi.


```

c memainkan kartu: kuning-8.

Giliran b.

true ?

yes
| ?- lihatKartu.

Berikut kartu yang anda miliki.
1. kuning-5

true ?

yes
| ?- mainkanKartu(1).

b memainkan kartu: kuning-5.

Permainan selesai! b menghabiskan semua kartunya!

Berikut perhitungan poin sisa kartu.
b: kartu habis = 0 poin
c: biru-1 + hitam-wild_draw_four + kuning-2 + hijau-3 = 26 poin
a: hijau-7 + hijau-2 = 9 poin

Urutan pemenang:
1. b (0 poin)
2. a (9 poin)
3. c (26 poin)

Selamat, b menjadi pemenang!

true ?

yes
| ?-

```

b) Dampak terhadap jalan permainan

Setelah permainan selesai, Program akan memberi urutan pemenang dari permainan yang telah selesai. Selain itu, program menghitung poin-poin yang didapatkan dari setiap pemain berdasarkan sistem perhitungan yang telah dijelaskan di atas. Pemain yang mendapatkan poin terendah ialah pemain yang memenangkan permainan UNI itu.

Pembagian Kerja

Nama Anggota	Tugas yang Dikerjakan
Christian Immanuel	Mengerjakan laporan, mengerjakan implementasi mainkanKartu, implementasi tangkap, dan implementasi util. Mengerjakan readme.md
Justin William	Mengerjakan laporan, mengerjakan implementasi ambilKartu, implementasi cekInfo, implementasiantang. Mengerjakan readme.md
Jeremy Gerald Sutanto	Mengerjakan laporan, mengerjakan implementasi startGame, implementasi semua actionCard, implementasi godsHand, implementasi mimic card, dan implementasi kartu disembunyikan.
Muhammad Ridwan Nasir Firdaus	Mengerjakan laporan, Mengerjakan implementasi lihatCommand, implementasi lihatKartu, implementasi saveGame, dan loadGame.

SURAT PERNYATAAN PENGGUNAAN KECERDASAN BUATAN/AI

Dengan ini, kami:

Nama/NIM : Justin William (13525042)
Muhammad Ridwan Nasir Firdaus (13525096)
Jeremy Gerald Sutanto (13525104)
Christian Immanuel (13525116)
Fakultas/Sekolah : STEI-K/Teknik Informatika
Kode Mata Kuliah : IF1221
Nama Mata Kuliah : Logika Komputasional
Judul : Praktikum Prolog
Tugas/Proposal

Menyatakan bahwa kami menggunakan AI/kecerdasan buatan dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan (level 1-5)
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya. Jelaskan secara singkat pemakaian dan <i>tool</i> yang digunakan:	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	-

Jelaskan secara singkat pemakaian dan <i>tool</i> yang digunakan:		
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya. Jelaskan secara singkat pemakaian dan <i>tool</i> yang digunakan:	Tidak	-
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya. Jelaskan secara singkat pemakaian dan <i>tool</i> yang digunakan:	Tidak	-
Lainnya Jelaskan secara singkat pemakaian dan <i>tool</i> yang digunakan: Kami menggunakan AI ChatGPT hanya untuk mempelajari bahasa pemrograman prolog seperti predikat dinamik, built-in predicate dan lain-lain. Tetapi kami tidak menggunakannya untuk mengerjakan proyek ini.	Ya	2

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 29 Mei 2026



Jeremy Gerald Sutanto
(13525104)